
Improving c-NCA: Toward a Virtual Stem Cell

September 11, 2026

Francesca Stefano, Tommaso Bellanova

1. Introduction

Neural Cellular Automata (NCAs) extend classical Cellular Automata with a differentiable and learnable local update rule. (Mordvintsev et al., 2020) take direct inspiration from Levin’s work on bioelectric signalling, in which collective, non-centralised communication among cells encodes target morphologies and drives their regeneration after damage (Levin, 2021). Mirroring this, they show that a single rule applied in parallel by all cells can grow a target pattern from a single seed and recover from damage. The global pattern emerges without central coordination, making NCAs a powerful model for self-organisation.

Our goal is to condition a Growing NCA to differentiate into various shapes based on an external signal, rather than growing a single fixed target. This serves as a computational analogy to stem cell differentiation: genetically identical cells acquire different fates depending on the ubiquitous signals they receive during growth.

A recent preprint (Sakour, 2025) explored a similar Conditional NCA (c-NCA) to grow MNIST digits. The work claims strong homeostatic stability and self-repair, despite using a simplified training protocol. By reproducing this baseline, we found that these properties do not hold under stricter evaluation. Reinstating the original NCA training protocol (pool and damage) significantly improves both properties without changing the underlying architecture.

We make three main contributions. First, we analytically show that, with a deterministic seed and an MSE loss, the only optimum is the class mean with zero variability; thus, training on the entire dataset is equivalent to training on ten fixed images (Appendix 1 6). Second, we evaluate three novel metrics not addressed in the preprint: plasticity under signal switching, interpolation regularity between class vectors, and the relationship between the learned filters and the fixed ones used in Growing-NCA. Third, we propose c-NCA v2, which combines the architecture of (Mordvintsev et al., 2020) with a better conditional mechanism respect of

the one of (Sakour, 2025). This last model achieved the best overall performances across all tests.

2. Background and related work

Growing NCA. In (Mordvintsev et al., 2020), a single shared local rule grows a target image starting from a single seed cell. The state of the automaton is represented by a grid where each cell has C channels: the first three for the RGB color, an α channel that indicates the “life” of the cell, and the remaining ones used as hidden channels. At each step, the cell builds its perception vector by applying a depthwise convolution to the channels using three fixed 3×3 filters: the identity and the two Sobel operators. The state update is calculated using a two-layer MLP:

$$\text{perceive}(x) = [x; S_x * x; S_y * x] \quad (1)$$

$$\Delta x = W_2 \text{ReLU}(W_1 \text{perceive}(x)) \quad (2)$$

The update is stochastic and constrained by a double *alive mask* $\alpha(\cdot)$, which sets to zero the channels of a cell if and only if its neighborhood is empty:

$$\tilde{x}_t = x_t + m_t \odot \Delta x_t \quad (3)$$

$$x_{t+1} = \tilde{x}_t \odot \alpha(x_t) \odot \alpha(\tilde{x}_t) \quad (4)$$

Applying the mask before the update is crucial to prevent dead cells with empty neighborhoods from activating spontaneously, breaking locality. Similarly to an RNN (which inspired the model), the training uses Backpropagation Through Time (BPTT) to calculate the gradients. To deal with the high memory consumption required by BPTT for long rollouts, a *pool* of historical states is used: at each step, 8 states are extracted; the worst one (highest loss) is re-initialized to the seed, while the best two are damaged to stimulate resilience. After evolving for a random number of steps in $[64, 96]$, the MSE loss is calculated on the visible channels.

Conditional morphogenesis. The approach of (Sakour, 2025) uses a single model to generate ten different digits, concatenating a one-hot vector e_k to the perception of each cell to simulate the conditioning:

$$\Delta x = W_2 \text{ReLU}(W_1 [\text{perceive}(x); e_k]) \quad (5)$$

Email: Francesca Stefano <stefano.2206334@studenti.uniroma1.it>.

This method differs from (Mordvintsev et al., 2020) in three key aspects: the perception filters are learned instead of fixed; only the alive mask $\alpha(\tilde{x}_t)$ is applied; the training happens in "de-novo" mode (64-step rollouts always starting from the seed, without using a pool or inducing damage); everything is repeated for 25 epochs. The damage resilience of the model was evaluated simply by setting to zero random cells.

3. Methods

First of all, we demonstrated that, with deterministic seed and conditioning and MSE loss, the only minimum of the objective corresponds to the class average generated without any variability (Appendix 1 (6)). Therefore, beyond reproducing the baseline, we trained and evaluated the models directly on these 10 fixed targets, comparing the effectiveness of the de-novo, pool, and pool+damage protocols. All these fixed-target training runs were conducted for 8000 steps using a batch size of 80 (8 states per class). For each model and training regime, we performed a series of tests:

Homeostasis. Fixing a time t_{ref} after the growth is complete, we quantify the long-term stability by calculating the MSE (compared to $x_{t_{\text{ref}}}$) and measuring the confidence of a LeNet-5 classifier on the states x_t , for extended evolutions up to $t = 4096$.

Damage resilience. To distinguish a real regenerative capacity from the simple memorization of the damages suffered during training (random disks), in the test phase we apply five unseen regional damages: setting to zero the top half, bottom half, right half, left half, or a central square. We then measure the residual error after a fixed regeneration time.

Plasticity. We grow the organism with the conditioning e_a for T_0 steps, and then switch it to e_b for another T_1 steps. By giving the final RGB averages as input to LeNet-5, we repeat the test on all the 90 possible class permutations to measure, as T_0 varies, how much the morphology can be reshaped in late stages.

Conditioning smoothness. Setting $f_\theta(c)$ as the final visible state generated with conditioning c , we linearly interpolate between two classes using $c(\lambda) = (1 - \lambda)e_a + \lambda e_b$. We then evaluate the norm of the directional derivative along the path, by estimating it with a centered difference quotient:

$$D_{ab}(\lambda) = \left\| \frac{df_\theta(c(\lambda))}{d\lambda} \right\| \approx \frac{\|f_\theta(c(\lambda + h)) - f_\theta(c(\lambda - h))\|}{2h}. \quad (6)$$

Low values of D_{ab} indicate smooth transitions between shapes, while peaks indicate critical points where the morphological change happens sharply.

Study of perception kernels. To check if the models spontaneously rediscover the spatial filters of the fixed baseline (Identity and Sobel filters), we measured the alignment between the subspace generated by the learned kernels and the reference one. Since basis change invariance prevents a direct comparison on individual filters, we calculated the shared dimensions using principal angles. Then we derived a p-value by comparing the results with randomly generated subspaces (mathematical details in Appendix 2 (6)).

Finally, we proposed another model (c-NCA v2). We applied three main changes to the baseline model of (Sakour, 2025): we restored the fixed Sobel filters, reintroduced the double alive mask, and adopted the training with pool and damage. Furthermore, we replace the concatenation with FiLM modulation (Perez et al., 2017). The local rule for the state increment Δx then becomes:

$$\Delta x = W_2 \text{ReLU}\left((1 + \gamma_c) \odot (W_1 \text{perceive}(x)) + \beta_c\right), \quad (7)$$

where the vectors γ_c and β_c are linearly generated starting from the class vector e_c . This formulation, at the cost of doubling the parameters for conditioning, clearly separates the roles: W_1 acts as a pure local spatial perception operator, while the class only modulates its output through FiLM. When $\gamma_c = 0$, (7) algebraically coincides with (5), showing its architectural consistency.

4. Experiments and Results

The experimental work follow these steps:

Homeostatic stability. First, we verified Sakour's results. The claimed homeostatic stability has been disproved. While (Mordvintsev et al., 2020) achieved almost unlimited stability over time, our tests up to 4096 steps showed that Sakour's generated digits degrade rapidly after just 128 steps. We measured this deviation using the mean squared error. To improve this, we replaced the *de novo* training with the same pooling strategy of Growing NCA, extending stability up to approximately 4096 steps. Conversely, our model demonstrates vastly improved stability compared to the others, scaling up to 16,000 steps and beyond. Additionally, ours is the only model that maintains an almost constant loss over time, whereas in the other models, the loss keeps growing at all time steps.

Table 2 shows how the reproduced baseline does not replicate the stability claimed in (Sakour, 2025), while the other model progressively recover both homeostatic stability and recovery capability, without sacrificing accuracy.

	MSE (RGBA)	LeNet-5 confidence
Base	/	/
Base (f.t)	/	/
Pool	0.0116	99.54%
Pool + Dam	0.3543	27.39%
Pool + Dam v2	0.0037	97.03%

Table 1. Homeostasis at t=4096 step

Damage Resilience. The second experiment focused on verifying the self-healing property against different kind of damage. In particular we let the organism evolve for T_0 steps, then we applied the damage and let it recover for T_1 steps. Our analyses, confirm this ability for the baseline in the case of stochastic damage, but struggles with more structured damage. Introducing damage during training significantly improves the performances in this regard. Our model exhibits damage resilience comparable to the pooling + damage approach. However, its distinguishing characteristics are a slower recovery process and a slight spatial translation and/or downscaling of the digits within the grid. In contrast, because the baseline model (with pooling + damage) lacks a dual alive mask, it occasionally introduces artifacts into the reconstructed digits.

	Recovery loss
Base	0.0107
Base (f.t)	0.0149
Pool	0.0112
Pool + Dam	0.0029
Pool + Dam v2	0.0038

Table 2. Damage analysis: $T_0, T_1 = 128$ for all modes excepts for Base and Base (f.t) where $T_0 = 64, T_1 = 48$.

Regularity in conditioning. We also investigated an aspect unexplored by the original author: how does the model react to a convex combination of condition vectors? The results are surprisingly robust for all model. At $T = 64$, we observed that as the interpolation parameter varies, the generated images transition smoothly from one digit to another. Notably, models trained with damage exhibit reduced smoothness, showing higher norm of the directional derivative.

	Max	Mean
Base	35.1 ± 16.8	15.6 ± 4.3
Base (f.t.)	40.6 ± 24.3	18.4 ± 5.2
Pool	39.7 ± 26.8	15.1 ± 3.5
Pool + Dam	85.9 ± 95.5	21.8 ± 6.7
Pool + Dam v2	60.8 ± 18.5	22.2 ± 5.2

Table 3. Mean and standard deviation of directional derivative at

fixed time $T=64$ step across all 45 unpaired pair.

Plasticity. We then conducted a *plasticity* test¹ by altering the condition vector of a fully formed digit to observe the "organism's" morphological adaptation. All the organism tested exhibits good plasticity if the new conditioning is applied at earlier steps, though this ability diminishes (in particular for models trained without damages) if the digit is left to stabilize for several steps. Notably, training with pooling and damage yields significantly higher tissue plasticity compared to just pooling. For the baseline model it was not possible to fully verify this property due to its severe instability; in contrast the v2 model, vastly outperformed all other model, as can be seen in Table 4.

$T_0 = 128, T_1 = 256$	
Pool	$0.1\% \pm 0.3\%$
Pool + Dam	$70.1\% \pm 30.8\%$
Pool + Dam v2	$92.4\% \pm 18.2\%$

Table 4. Mean and standard deviation of LeNet-5 probability of recognizing the second digit across all 90 pairs after the conditioning change (6).

Learned Kernels. Another interesting aspect concerns the convolutional perception network. Unlike (Mordvintsev et al., 2020), who relies on fixed Sobel and identity filters, (Sakour, 2025) employs learned perceptual kernels. Comparing the subspaces generated by these two sets of filters, we found that the model learns to perceive using features that are substantially different from the fixed ones. For mathematical details 6.

p-value < 0.05	
Base	4/16
Base (f.t.)	4/16
Pool	4/16
Pool + Dam	6/16

Table 5. Learned kernels, number of channels with p-value < 0.05

5. Discussion and Conclusions

In conclusion our findings refute Sakour's claims about stability and damage resilience, under original training regime. By adding modifications that are supported by literature, we demonstrated a substantial recovery of these key properties. We found out other important facts:

- Using all the MNIST training set was pointless due

¹Plasticity is the ability of a committed tissue or cell to adopt a new identity upon stimulation.

to the architecture of the model and the choice of the loss.

- The learned filters doesn't seem to be particularly related to the fixed ones used in Growing-NCA as biological prior.
- The introduction of damage during training also improves dramatically the plasticity of the organism.

Furthermore our new c-NCA v2 model achieve impressive homeostatic stability, regenerative capabilities and plasticity if compared to the v1 model. We think that the improvement in homeostasis are due to the double alive mask, and we suspect that the higher plasticity depends also on FiLM conditioning, however more experiments are needed to falsify those claims. Together, these changes provide a more stable foundation for a conditional model, bringing its behavior closer to the biological analogies that inspired it.

6. Statement on the use of AI

For this project, we made extensive use of LLMs. The principal uses can be resume in those areas:

- **Coding and debugging:** In terms of implementation of the core codebase, which included conditional NCA architectures, training loops, and evaluation metrics such as homeostasis and kernel analysis, we basically left little to no autonomy to the AI. In that case, the AI could not make architectural or logical decisions but was merely used to implement the algorithmic instructions provided by us in full detail and literally translate them into code. Each line of code produced by the AI was carefully studied and validated. In turn, we made extensive use of the AI to overcome our lack of knowledge concerning certain utility libraries (specifically, SciPy, Matplotlib, and Pandas) and help us with data visualization and statistical pipeline setup. Additionally, we used the AI extensively during the process of the code refactoring to advise on the proper structuring of the code to keep it maintainable and debuggable.
- **Writing, Formalization, and Literature Search:** For the reports, LLMs were heavily utilized to transform our Italian drafts into academically sound English and produce text to strictly adhere to the page limits required for submission. Moreover, the AI was used to help us in the formalization of Appendices 1 and 2. In particular it helped us to describe the interaction between the perception kernels and the MLP network and the proof of the theorem of uniqueness.

In Appendix 2 (and when writing the code) it explained to us how to estimating p-values from principal angles. In terms of finding other techniques of conditioning besides concatenation, the AI recommended Feature-wise Linear Modulation (FiLM) and directed us to its seminal work (Perez et al., 2017), which served as a direct inspiration for our c-NCA v2 model. paper (Perez et al., 2017), which directly inspired the architecture of our proposed c-NCA v2 model.

All code generated by an AI tool, explanations, and text have been reviewed and comprehended by us prior to being submitted, and we assume full responsibility for its correctness and contents.

References

- Levin, M. Bioelectric signaling: Reprogrammable circuits underlying embryogenesis, regeneration, and cancer. *Cell*, 184(8):1971–1989, 2021.
- Mordvintsev, A., Randazzo, E., Niklasson, E., and Levin, M. Growing neural cellular automata. *Distill*, 2020. doi: 10.23915/distill.00023.
- Perez, E., Strub, F., de Vries, H., Dumoulin, V., and Courville, A. Film: Visual reasoning with a general conditioning layer, 2017. URL <https://arxiv.org/abs/1709.07871>.
- Sakour, A. Conditional morphogenesis: Emergent generation of structural digits via neural cellular automata. *arXiv preprint arXiv:2512.08360*, 2025.

Appendix

Appendix 1: Fixed target

Sakour presents c-NCA as a class-conditioned generative model. We disagree: we show below that the objective actually minimized during its training has a unique minimum, which is the organism that deterministically reproduces the class mean. Hence the choice of fixed targets, adopted in all subsequent experiments: if the only reachable optimum is the class mean, we might as well use it as an explicit target. This way, the training loss measures the model's error and not the intra-class variance, which no model of this family can capture.

Let $\mathcal{T} = \{T_1, \dots, T_N\} \subset \mathbb{R}^D$ be the training set. All the randomness inherent in the training set comes from the data loader, which we model for simplicity as a random variable $I \sim \text{Unif}([N])$, therefore, at each step, we randomly draw a target $T = T_I$. Let $K = K_I$ also be the random variable indicating the class of the drawn target (e.g., the MNIST label).

Let $f_\theta(K, \omega) \in \mathbb{R}^D$ be the output produced by our model instead (in our case, we can think of it as the entire grid on the visible channels after a fixed rollout time). ω is the variable that contains all the model's randomness (e.g., stochastic update mask, pool sampling, random damage, etc.).

Up to a factor of $1/D$ the quantity minimized during training is an estimate of:

$$L(\theta) = \mathbb{E}[(f_\theta - T)^2].$$

Where the expectation is calculated over the random vector (I, ω) and, by an abuse of notation, we set $(f_\theta - T)^2 = \|f_\theta - T\|^2$.

We assume the following hypothesis:

$$(H_1): \quad f_\theta \perp T \mid K = k.$$

Namely, that f_θ and T are conditionally independent given K . This hypothesis holds true in our setting, since the randomness of the c-NCA model ω does not depend in any way on the specific drawn target T .

For a random vector $X \in \mathbb{R}^D$ we write

$$\text{Var}(X|k) = \mathbb{E}[(X - \mathbb{E}[X|k])^2|k],$$

that is, the sum of the variances of the D components. All the sources of randomness involved are discrete and finite (the index I , the asynchronous update mask, the pool sampling index, the damage mask), so the second moments exist and the following equalities hold pointwise, for every realization with positive probability.

Lemma 1. *Assuming H_1 , the following decomposition holds:*

$$L(\theta|k) = (\mathbb{E}[f_\theta|k] - \mathbb{E}[T|k])^2 + \text{Var}(f_\theta|k) + \text{Var}(T|k)$$

Proof. The idea is to make the two variances appear by adding and subtracting the two means. We write

$$\begin{aligned} f_\theta - T &= (f_\theta - \mathbb{E}[f_\theta|k]) - (T - \mathbb{E}[T|k]) \\ &\quad + (\mathbb{E}[f_\theta|k] - \mathbb{E}[T|k]), \end{aligned}$$

which is an identity, because the added terms cancel each other out.

Squaring both sides yields the squares of the three terms and their three cross-terms. The three squares, taken in expectation, are exactly the three terms of the statement: the variance of f_θ , the variance of T , and the squared difference between the two means, which is already deterministic once the class is fixed. It remains to be seen that the cross-terms are all zero.

Two of the three contain the factor $\mathbb{E}[f_\theta|k] - \mathbb{E}[T|k]$. This is a fixed vector once the class is chosen, so it can be taken out of the expectation; and what remains inside is a centered variable, which has a zero mean by construction.

The third is the cross-term between the two centered variables, and this is the only point where (H_1) is needed. Looking at it component by component, each term is the product of a quantity depending only on f_θ and one depending only on T . Conditional independence then allows splitting the expectation of the product into the product of the expectations, $\mathbb{E}[XY|k] = \mathbb{E}[X|k]\mathbb{E}[Y|k]$, and since both quantities are centered, the result is zero. $\square$

Averaging over the class, with $p_k = \mathbb{P}(K = k)$, and setting

$$L^* = \sum_k p_k \text{Var}(T|k),$$

the quantity actually minimized is

$$\begin{aligned} L(\theta) &= \sum_k p_k L(\theta|k) \\ &= \sum_k p_k [(\mathbb{E}[f_\theta|k] - \mathbb{E}[T|k])^2 + \text{Var}(f_\theta|k)] \\ &\quad + L^*, \end{aligned}$$

where L^* does not depend on θ .

Theorem 2. *Assuming H_1 , for every θ we have $L(\theta) \geq L^*$, and the equality holds if and only if for every class k with $p_k > 0$ and for every realization ω*

$$f_\theta(k, \omega) = \mathbb{E}[T|k].$$

Proof. Of the three terms in the decomposition, two depend on θ : the squared difference between the means and the variance of f_θ . Neither can be negative, so $L(\theta)$ never falls below L^* .

For the equality to hold, both must be zero, and for every class k appearing in the training set: since all terms in the sum are non-negative and the weights p_k are positive, it is enough for just one to remain positive for the total to be positive.

Let us see what each of the two requires. A zero variance means that $f_\theta(k, \cdot)$ does not change as ω varies, meaning that the model, once the class is fixed, always produces the same output; and since ω takes a finite number of values, "always" is to be understood literally, on every realization. The zero difference then tells us what this single output is: the mean $\mathbb{E}[T|k]$.

Conversely, if the model always produces $\mathbb{E}[T|k]$, both terms cancel out and the loss is exactly L^* . $\square$

Operational consequence. The theorem identifies $\mathbb{E}[T|k]$, so we decide to use as targets ten fixed images, one per class, defined channel by channel as $\mathbb{E}[T|k]$ (grayscale mean replicated across the three RGB channels, mean of the binary masks on the alpha channel). In this regime, $L^* = 0$ and the training loss measures only the model's error.

Appendix 2: The learned filters and the Sobel subspace

In Growing NCA, Mordvintsev fixes the perception step to the three filters I , sobel_x , sobel_y . It is natural to ask whether the models that learn their perception filters rediscover those same filters, at least to some extent. This appendix explains why the comparison cannot be done filter by filter, which hypothesis is needed for it to be well posed, and how the p-value was computed.

Let us fix the notation. For each channel c of the v1 model, let $K_c \in \mathbb{R}^{3 \times 9}$ be the matrix whose rows are the three filters learned on that channel, flattened; let $V_c \in \mathbb{R}^{H \times 3}$ be the block of three columns of the first layer of the update network that reads the perception of channel c ; and let

$$A_c = V_c K_c \in \mathbb{R}^{H \times 9}$$

be the effective map that takes the 3×3 neighborhood of channel c into the hidden layer.

Why we cannot compare the matrices. The network does not see K_c and V_c separately, it only sees their product. For every invertible matrix $M \in GL(3)$,

$$(V_c M^{-1})(M K_c) = A_c,$$

so the pair (V_c, K_c) is determined only up to an arbitrary 3×3 change of basis (in the domain of V_c and in the codomain of K_c), and the loss has no a priori reason to prefer the canonical basis. As a consequence, the single learned filters are not identifiable a priori.

What we can study instead is the subspace spanned by the rows, $\text{Row}(K_c) \subset \mathbb{R}^9$, because it does not change under a change of basis in the codomain. The idea is therefore to compare $\text{Row}(K_c)$ with $\text{Span}\{I, \text{sobel}_x, \text{sobel}_y\}$.

Why we need rank 3. The rows of A_c are linear combinations of the rows of K_c , hence

$$\text{Row}(A_c) \subseteq \text{Row}(K_c)$$

always holds. Equality holds if and only if V_c has full rank, and in that case studying $\text{Row}(K_c)$ is legitimate, because it is the same as what the dynamics really uses. If instead $\text{rank}(V_c) < 3$, one direction of the perception subspace is computed and then set to zero by the next layer: $\text{Row}(K_c)$ overstates what the model uses, and comparing it with a three-dimensional subspace is ill posed. In the same way, if $\text{rank}(K_c) < 3$ the three filters are linearly dependent and the channel extracts fewer than three features, so the subspace we want to compare is not three-dimensional.

On the other hand we have

$$\text{rank}(A_c) \leq \min\{\text{rank}(V_c), \text{rank}(K_c)\} \leq 3,$$

so it is enough to check that $\text{rank}(A_c) = 3$; and the condition is also necessary, because if both ranks are 3 then V_c has full column rank and A_c has rank 3.

Since we are working at the level of numerical linear algebra, what matters is how full the rank is in a numerical sense. In particular we want to study how flat (or not) the ellipsoid $A_c(S^8)$ is, that is, the ellipsoid we obtain by applying A_c to the unit sphere of $\mathbb{R}^9$. Six axes of that ellipsoid are zero by construction (the directions in the kernel of A_c , which has dimension at least 6), namely the ones the channel ignores, because out of the nine pixels it extracts at most three features. The question is therefore whether the three remaining axes are all non-degenerate, that is, whether the ellipsoid is a three-dimensional body and not a flat disc or a segment. We therefore compute the ratio σ_1/σ_3 , where σ_1 is the longest axis of the ellipsoid and σ_3 is the shortest among the three axes that can be non-zero. Full rank is also the condition that makes the orthonormal basis of $\text{Row}(K_c)$ we use below well defined.

Principal angles and shared dimensions. Let $Q_{\text{ref}}, Q_c \in \mathbb{R}^{9 \times 3}$ be orthonormal bases of the reference space $\text{Span}\{I, \text{sobel}_x, \text{sobel}_y\}$ and of $\text{Row}(K_c)$. The cosines of the three principal angles between the two

subspaces are the singular values of $Q_{\text{ref}}^\top Q_c$, and we define

$$d_c = \sum_{i=1}^3 \cos^2 \theta_i = \|Q_{\text{ref}}^\top Q_c\|_F^2 = \text{tr}(P_{\text{ref}} P_c),$$

where P denotes the orthogonal projector onto the corresponding subspace. We call d_c the number of *shared dimensions*: it is 3 when the two subspaces coincide and 0 when they are orthogonal. The third expression shows that d_c does not depend on the bases we choose, which is exactly the property required by the previous paragraph.

The p-value. We want a test that answers the question: is the overlap observed on channel c larger than the one we would get by pure chance?

We state the null hypothesis H_0 : the three filters of channel c are three arbitrary directions of $\mathbb{R}^9$, with no relation to the reference subspace. To make "arbitrary" operational we ask that no direction be privileged, and this is exactly what Gaussians do. Indeed, a vector with nine independent $N(0, 1)$ components has a law that is invariant under rotations, so it does not point anywhere in particular. We therefore simulate H_0 by generating three such vectors, computing the shared dimensions of the subspace they span, and repeating $N = 20000$ times.

The mean under H_0 can be obtained in closed form: by the same invariance, the projector onto a random three-dimensional subspace of $\mathbb{R}^9$ has mean $\mathbb{E}[P] = \frac{3}{9}\mathbb{I}$, hence

$$\mathbb{E}[d] = \text{tr}(P_{\text{ref}} \mathbb{E}[P]) = \frac{3}{9} \text{tr}(P_{\text{ref}}) = 1.$$

The tail (and therefore the p-value) has no convenient closed form, and this is what the simulation is needed for. The right-tailed p-value is then the fraction of random subspaces that did at least as well as the channel,

$$p_c = \frac{1 + \#\{d_{H_0} \geq d_c\}}{1 + N},$$

where adding 1 to the numerator and to the denominator is the standard form of the p-value estimator.

From the tests we ran, for all 4 models with learned kernels we found $\text{rank}(A_c) = 3$ (in the numerical sense we discussed), and for each of them we report the number of channels with $p_c \leq 0.05$.

Appendix 3: Images

HOMEOSTATIC STABILITY

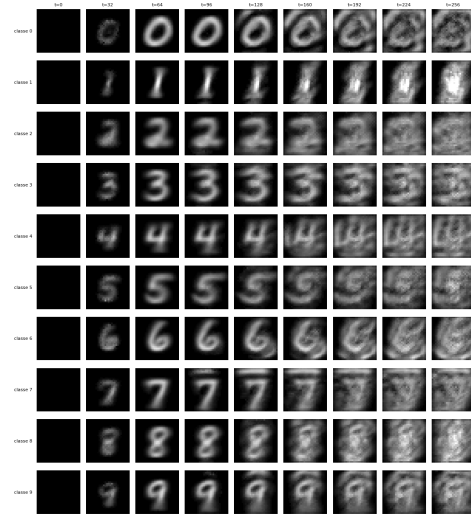


Figure 1. Homeostatic stability, model: baseline (max= 256 step)

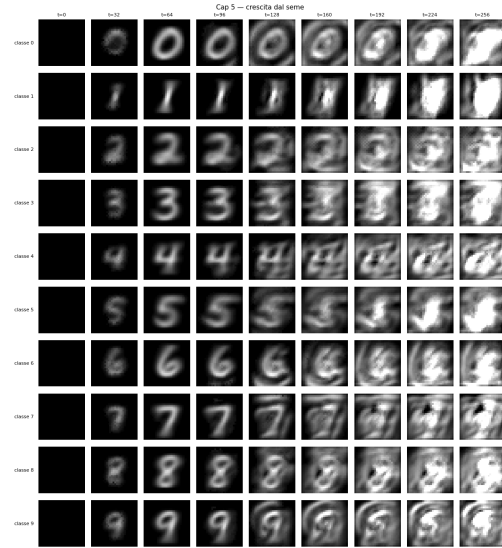


Figure 2. Homeostatic stability, model: baseline with fixed target (max= 256 step)

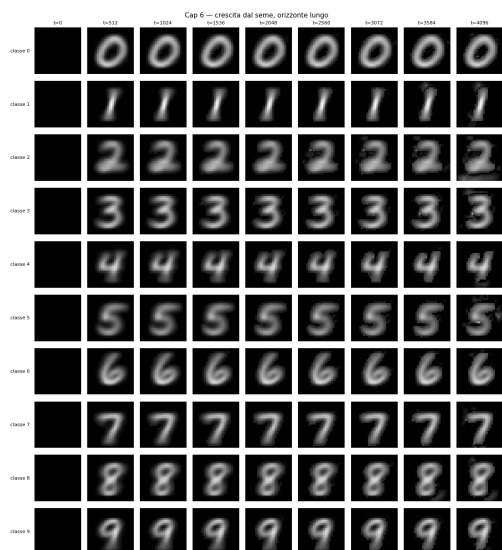


Figure 3. Homeostatic stability, model: pooling (max= 4096 step)

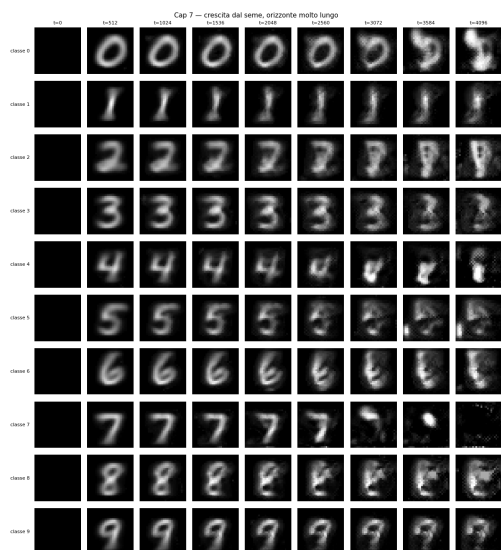


Figure 4. Homeostatic stability, model: pool+damage (max= 4096 step)

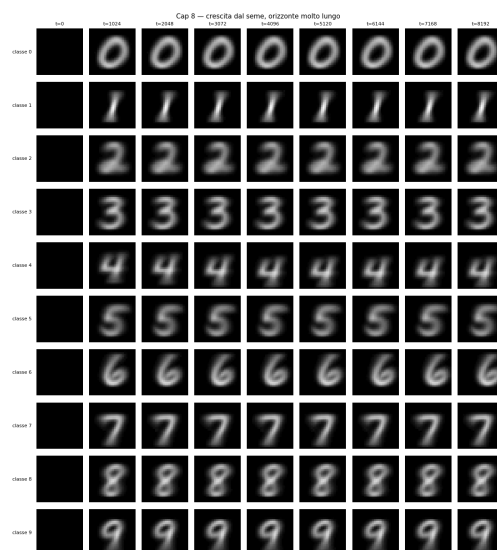


Figure 5. Homeostatic stability, model: v2 (max= 8192 step)

Here we show the process of plasticity for the couple $3 \rightarrow 1$

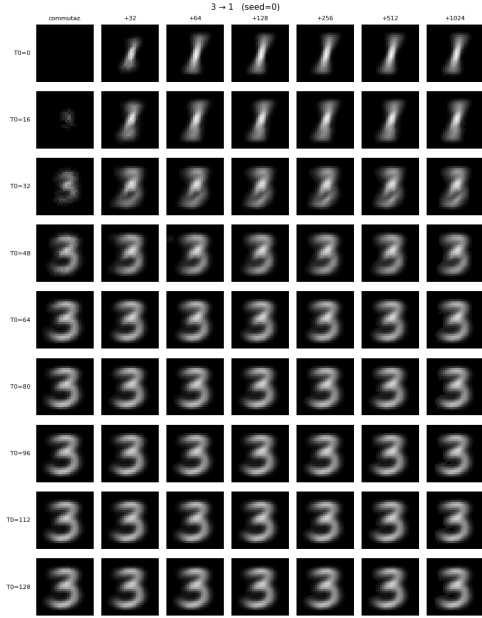


Figure 6. Plasticity, model: pooling

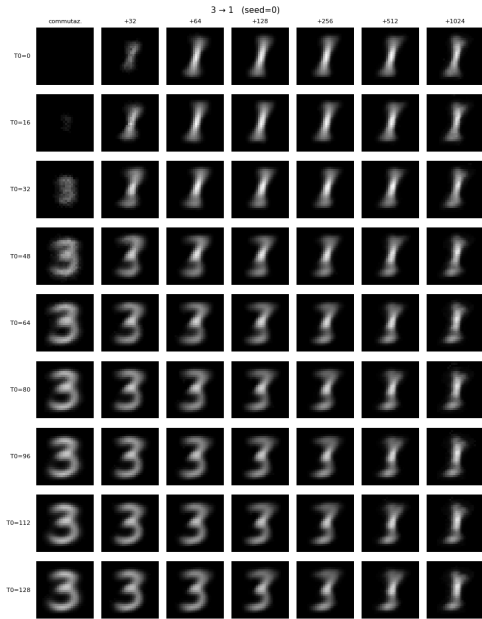


Figure 7. Plasticity, model: pooling + damage

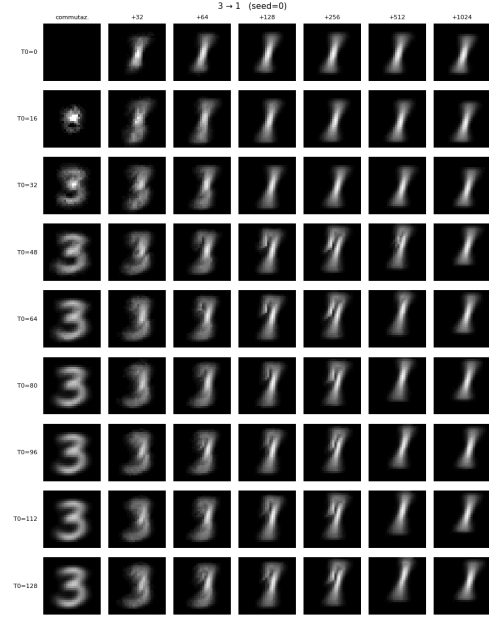


Figure 8. Plasticity, model: v2

Appendix 4: Testing on RGB 64x64 Images

Ultimately, we tested our model on a dataset of ten synthetic 64×64 RGB images. Scaling to this regime required adjustments to the c-NCA v2 pipeline. Targets are alpha-premultiplied as in Growing NCA, mapping empty space and dead cells to $(0, 0, 0, 0)$ and naturally weighting color error by opacity, $\alpha^2 \|\hat{c} - c\|^2$. We enlarged the batch from 80 to 100 (10 states per class) and trained for 40 “epochs”² of 400 iterations each. The rollout range shifted to $[80, 112]$ steps: with $\text{fire_rate} = 0.5$ and a 32-cell radius to the corner, at least 64 steps are expected just to fill the canvas.

Due to BPTT memory usage, we introduced micro-batching (rescaling chunk losses by n/N before accumulating gradients) and a cosine-annealed learning rate ($\text{lr} \rightarrow \text{lr}/10$) to sustain convergence. Damage radii remain specified as area fractions, applying the same relative damage as at 28×28 .

Despite these adaptations, the generated organisms are blurrier than their MNIST counterparts. This is not a resolution or capacity limit, an unconditioned NCA reproduces a single 64×64 target sharply, but likely stems from inter-class competition and insufficient training time³. By training with pooling and damage strategies, we achieved overall acceptable homeostasis and resilience, except for some classes like Spongebob.

²In this context, epochs mean just a certain number of training steps.

³However, even in later epochs, image details kept improving.



Figure 9. Homeostatic stability(max=512 step)

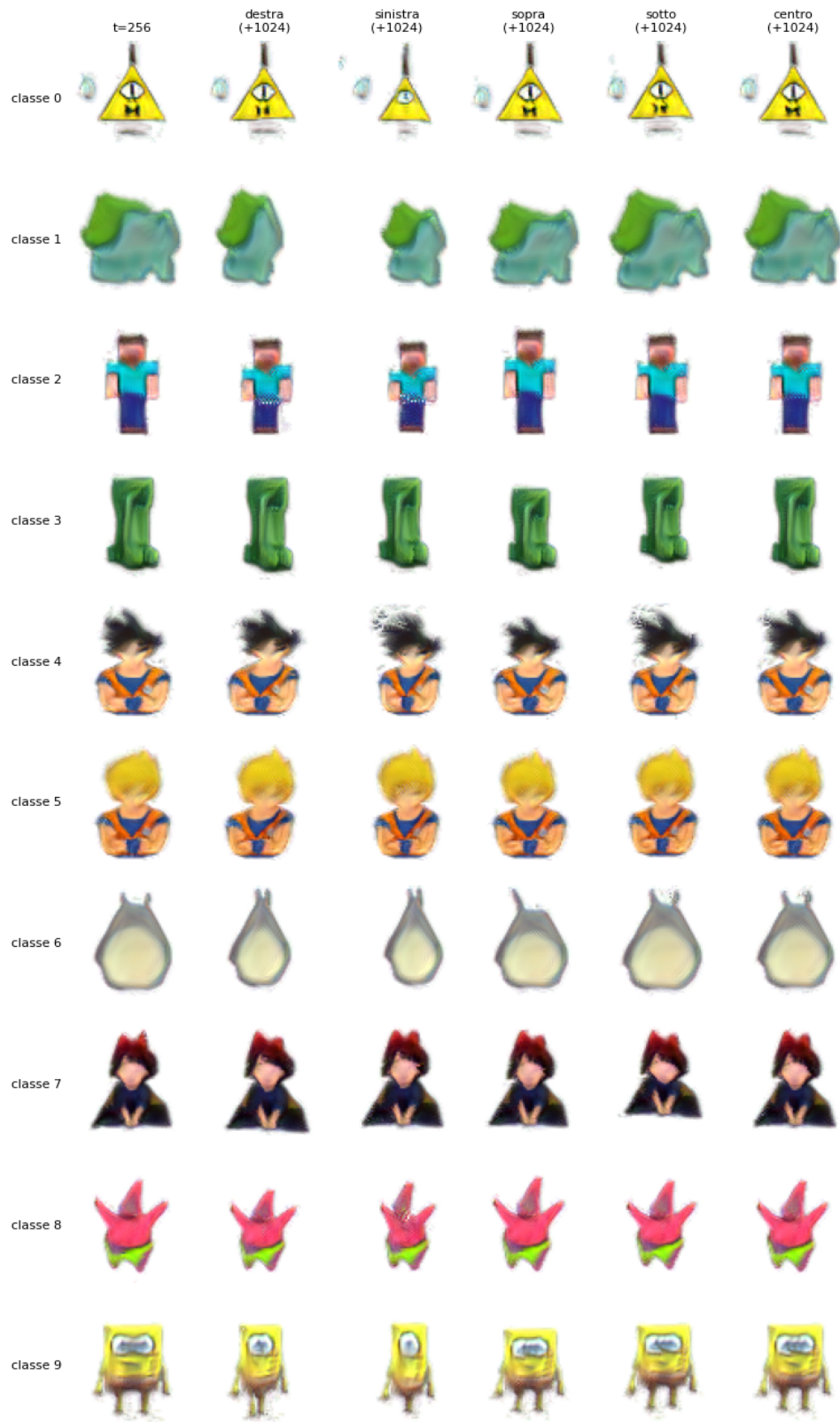


Figure 10. Localized damage

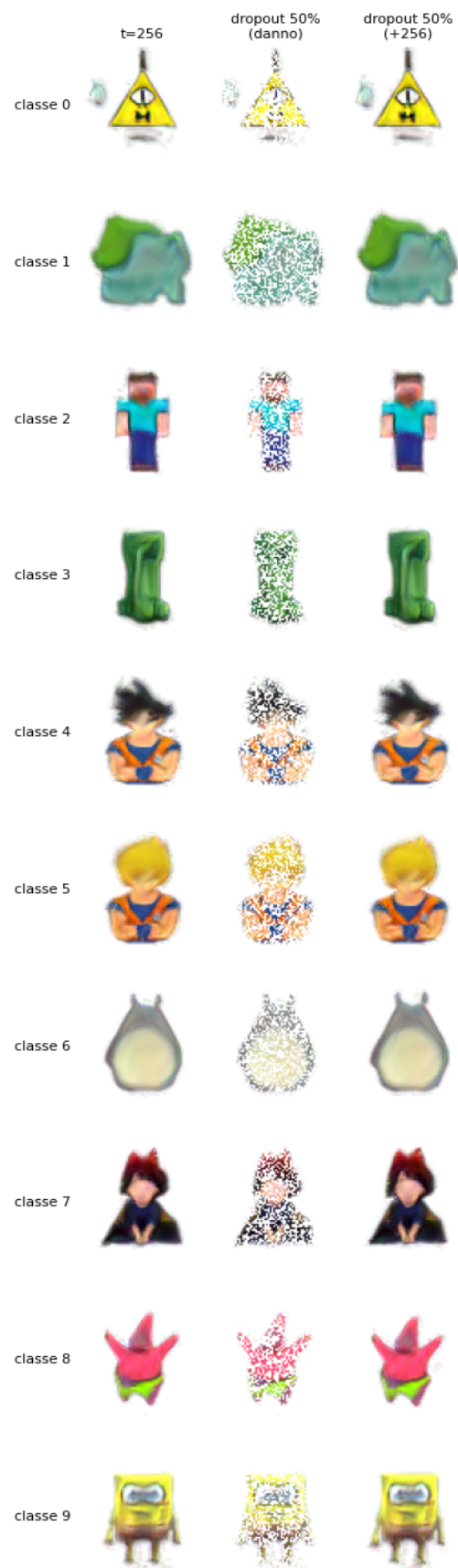


Figure 11. Random damage